Objects, Inheritance, and Linked Lists

Computer Science Mentors 61A

October 21 – October 24, 2025

1 Inheritance and Representation

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1. **Flying the cOOP** What would Python display? Write the result of executing the code and the prompts below. If a function is returned, write "Function". If nothing is returned, write "Nothing". If an error occurs, write "Error".
Hint: You may find it helpful to make an environment diagram tracking your objects and *each* new instance of an object (whether it's assigned to a variable or not!).

```
class Bird:
    def __init__(self, call):
        self.call = call
        self.can_fly = True
    def fly(self):
        if self.can_fly:
            return "Don't stop me
                now!"
        else:
            return "Ground control to
                Major Tom..."
    def speak(self):
        print(self.call)

class Chicken(Bird):
    def speak(self, other):
        Bird.speak(self)
        other.speak()

class Penguin(Bird):
    can_fly = False
    def speak(self):
        call = "Ice to see you"
        print(call)

andre = Chicken("cluck")
gunter = Penguin("noot")
```

```
>>> andre.speak(Bird("coo"))
```

```
>>> andre.speak()
```

```
>>> gunter.fly()
```

```
>>> andre.speak(gunter)
```

```
>>> Bird.speak(gunter)
```

2. What would Python display? Write the result of executing the following code and prompts. If nothing would happen, write "Nothing". If an error occurs, write "Error".

```
class ForceWielder():
    force = 25

    def __init__(self, name):
        self.name = name

    def train(self, other):
        other.force += self.force / 5

    def __str__(self):
        return self.name

class Jedi(ForceWielder):
    lightsaber = "blue"

    def __str__(self):
        return "Jedi " + self.name

    def __repr__(self):
        return f"Jedi({repr(self.name)})"

class Sith(ForceWielder):
    lightsaber = "red"
    num_sith = 0

    def __init__(self, name):
        super().__init__(name)
        Sith.num_sith += 1
        if self.num_sith != 2:
            print("Two there should be. No more, no less.")

    def __str__(self):
        return "Darth " + self.name

    def __repr__(self):
        return f"Sith({repr(self.name)})"
```

```
>>> anakin = Jedi("Anakin")
>>> anakin.lightsaber, anakin.force


>>> obiwan = Jedi("Obi-wan")
>>> anakin.master = obiwan
>>> anakin.master


>>> Jedi.master


>>> obiwan.force += anakin.force
>>> obiwan.force, anakin.force


>>> obiwan.train(anakin)
>>> obiwan.force, anakin.force


>>> Jedi.train(obiwan, anakin)
>>> obiwan.force, anakin.force


>>> sidious = Sith("Sidious")


>>> ForceWielder.train(sidious, anakin)
>>> anakin.lightsaber = "red"
>>> anakin.lightsaber, anakin.force


>>> Jedi.lightsaber


>>> print(Sith("Vader"), Sith("Maul").num_sith)


>>> rey = ForceWielder("Rey")
>>> rey


>>> rey.lightsaber
```

2 Linked Lists

1. What will Python output? Draw box-and-pointer diagrams along the way.

```
>>> a = Link(1, Link(2, Link(3)))
>>> a.first


>>> a.first = 5
>>> a.first


>>> a.rest.first


>>> a.rest.rest.rest.rest.first


>>> a.rest.rest.rest = a
>>> a.rest.rest.rest.rest.first


>>> repr(Link(1, Link(2, Link(3, Link.empty))))


>>> Link(1, Link(2, Link(3, Link.empty)))


>>> str(Link(1, Link(2, Link(3))))


>>> print(Link(Link(1), Link(2, Link(3))))
```

2. Write a function `reverse`, which takes in a `Link` and returns a new `Link` that has the order of the contents reversed.
   Hint: You may want to use a helper function if you're solving this recursively.

```
def reverse(lst):
    """
    >>> a = Link(1, Link(2, Link(3)))
    >>> b = reverse(a)
    >>> b
    Link(3, Link(2, Link(1)))
    >>> a
    Link(1, Link(2, Link(3)))
    """
```

3. Write a recursive function `insert_all` that takes as input two linked lists, `s` and `x`, and an index `index`. `insert_all` should return a new linked list with the contents of `x` inserted at index `index` of `s`.

```
def insert_all(s, x, index):
    """
    >>> insert = Link(3, Link(4))
    >>> original = Link(1, Link(2, Link(5)))
    >>> insert_all(original, insert, 2)
    Link(1, Link(2, Link(3, Link(4, Link(5)))))
    >>> start = Link(1)
    >>> insert_all(original, start, 0)
    Link(1, Link(1, Link(2, Link(5))))
    """
    if _____________________ and _____________________:

        _____________________________________________

    if _____________________ and _____________________:

        _____________________________________________

    _____________________________________________
```